RAY TRACING ON GRAPHIC PROCESSORS: TOWARDS HIGH FIDELITY RADIATIVE TRANSFER SOLVERS

Faizan P. Siddiqui, Altug M. Basol, M. Pinar Mengüç
Mechanical Engineering Department,
Ozyegin University, Istanbul, Turkey

ABSTRACT: The extreme computational cost of Monte Carlo ray tracing method precludes its widespread use for the numerical solution of radiative transfer equation. This work focuses on substantially decreasing the computational cost of the method by utilizing the computation power of the modern graphic processors. Running ray tracing computations for view factor calculations on a high end GPU resulted in a 40x increase in the number of rays being traced compared to the similar studies found in literature. In addition to the rise of the computational power purely by hardware, the usage of the acceleration data structure on the computational performance has also been separately investigated which resulted in about a 10x speed up on top of the speed up gained entirely by GPU.

INTRODUCTION

One of the ways of solving radiative transfer equation especially in non-participating medium relies on the calculation of view factors between surfaces. The calculated view factors are later on used for the evaluation of surface-to-surface thermal radiation exchange. Analytical solutions of view factors calculations exist only for regular geometries. A comprehensive catalog for these analytical solutions have been provided by Howell [1] and Howell and Mengüç [2,3]. However, view factors for complex irregular geometries can only be calculated numerically. Nusselt's unit sphere method [4], Farrell's optical projection method [5], Hottel's Cross-string method [6] and Monte-Carlo ray tracing method [7] are some of them. A study on the comparison of these numerical techniques, presented by Emery et al. [8], revealed the advantage of the Monte Carlo ray tracing method in terms of accuracy. Similarly, Hajji et al. [9] and Mirhosseini and Saboonchi [10] pointed out the accuracy of the Monte Carlo ray tracing method and they both have highlighted its prohibitively high computational cost preventing its widespread use. Yet, Monte Carlo approaches can be computationally very expensive among the others considered if they are not implemented with care.

Apart from the view factor calculations for the solution of the radiative transfer equation, ray tracing is also being widely used in computer graphics. The application of ray tracing in computer graphics has been first carried out by Whitted [11] back in 1980. To generate the image of a three-dimensional object, rays from an imaginary camera is emitted in the directions of the pixels of an

imaginary screen where the image will be formed. The pixels are colored depending on the positions the rays hit on the scene behind the screen. In principle, it works in a similar fashion a pinhole camera generates images but in this method rays are traced backward from the camera to the objects and also the screen is placed between the camera and the object. The success of this method generating high quality images led to further developments. Kajiya [12] combined ray tracing with the Monte Carlo method to include the effect of the indirect lighting into the images. As opposed to the ray tracing technique applied by Whitted [11], Kajiya [12] introduced randomness to the paths rays have followed. The so called "path tracing" method is able to generate photorealistic renderings of the objects. However, the path tracing method requires millions of rays to be traced which is computationally very costly. Due to this reason ray tracing type image rendering is mostly carried out in large computer clusters as batch jobs.

Finding ray-object intersection in a large complicated scenario is computationally expensive task. Without acceleration structures each ray has to be tested with all objects present in the domain. For a scene containing 'N' number of triangles and using 'M' number of rays, the computational complexity will be N×M. Since in Monte Carlo implementation generally large number of rays are required to get statistically accurate results, so the practical cases which involve complex geometries, would become enormously expensive. To overcome this difficulty efficient non-linear data structures are used to accelerate the computation. Most popular techniques include KD-tree [13] and Bounding volume hierarchy [14]. A detailed study has been conducted by Vinkler et al. [15] on the performance comparison of Kd Tree and Bounding volume hierarchy implemented on multi core processors.

Graphic cards are the components of the computer that are entirely devoted to calculations related to image generation. Over time the available computational power has increased by many folds resulting in more realistic animations in computer games. The processing unit on the graphic card, the graphic processing unit (GPU), is developed in particular for the needs of the computer graphics and in that sense they suit best for massively parallel applications. However, due to their inherent hardware architecture the performance of the GPU heavily depends on the way it is programmed. It is quite possible to develop an application running on GPU with lower performance than it does on CPU. So, the usage of the efficient code libraries is crucial for the final performance of the developed tools. In this regard, Nvidia has released the programmable ray tracing engine called OptiX [16] to be used to develop ray tracing based applications running efficiently on GPU. Even though this engine is particularly developed for game and computer graphics applications it can also be utilized for ray tracing based radiative transfer equation solvers. In this regard, Halverson [17] used OptiX ray tracing engine for solar radiation modeling of an urban area.

In this study a Monte Carlo ray tracing method based view factor calculation software has been developed utilizing Nvidia's Optix [16] ray tracing engine. The tool has first been validated and afterwards the performance of the tool is compared with the recent studied found in literature. The gain in the computation performance with the developed tool is discussed in detail by separating

the effects of the computational power of the modern GPU's and the influence of the bounding volume hierarchy (BVH) on the observed performance.

NUMERICAL METHOD

Present work is focused in calculating view factors for complex, three-dimensional objects which are represented by the triangulated surface mesh data. Accordingly, the view factor calculations are carried out for every triangle on the surface of the object by emitting a ray from the center as well as at different random location within the triangles. In this manner a view factor map is generated. The direction on which the ray is emitted is determined by its azimuthal ($\emptyset$) and zenith angles (θ) as explained by Howell et al. [18]. The zenith angle θ is calculated by using equation (1), where R_1 is a uniformly distributed random number between 0 and 1. Since Monte-Carlo method is stochastic process, random number generation plays an important role for the correct implementation of this method. In this regard, Tiny Encryption Algorithm [19] has been used for generating sequences of random numbers on graphic processor.

$$\theta = \sin^{-1}\sqrt{R_1} \tag{1}$$

The azimuthal angle $\emptyset$ is calculated randomly as well generating another random number R_2:

$$\emptyset = 2\pi R_2 \tag{2}$$

Finally, the direction vector p is formed using zenith and azimuth angles as given below.

$$p = \begin{bmatrix} \sin\theta\ \cos\emptyset \\ \sin\theta\ \sin\emptyset \\ \cos\theta \end{bmatrix} \tag{3}$$

Rays are emitted from determined emission points and angles and traced in the domain. Ray-object intersections are next determined. The Optix [16] library is especially optimized to deal with this computationally expensive task, to calculate a view factor using

$$F_{1-2} = \frac{I}{J} \tag{4}$$

where *I* is the number of rays hitting the object and *J* is the total number of rays emitted from the surface of the source.

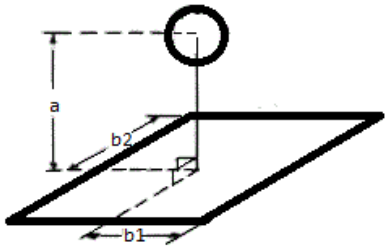

Figure 1. Test case for calculating view factor [1].

Validation of the Method:

For the validation of the numerical method one of the view factor scenarios published by Howell [1,2] is selected. As sketched in Figure 1, the scenario involves the view factor calculation between the sphere and the plate. The analytical solution given below was derived by Feingold and Gupta [20].

$$F_{1-2} = \frac{1}{2\pi}\left\{\sin^{-1}\left[\frac{2B_1^2 - \left(1 - B_1^2\right)\left(B_1^2 + B_2^2\right)}{\left(1 + B_1^2\right)\left(B_1^2 + B_2^2\right)}\right] + \sin^{-1}\left[\frac{2B_2^2 - \left(1 - B_2^2\right)\left(B_1^2 + B_2^2\right)}{\left(1 + B_2^2\right)\left(B_1^2 + B_2^2\right)}\right]\right\} \quad \text{where B1=b1/a B2=b2/a} \qquad (5)$$

To compute the view factor numerically rays have been emitted from the surface of the sphere and traced through the domain as explained in the numerical method section. The view factor calculations have been conducted for varying distances between the sphere and the plate. For the calculations at each distance value, 0.275 million rays were used. In Figure 2 the numerically predicted view factors as well as the analytical solution are plotted with respect to the dimensionless distance between the two objects. As shown, the numerical values match reasonably well with the analytical solution.

Due to its stochastic nature in Monte-Carlo method the outcomes of the calculations vary from one trial to the other. Figure 3 shows the histogram of the relative errors between the numerical and analytical value of the view factor. The distribution is obtained by repeating the view factor calculations for a single distance value thousand times. Each set of calculations were carried out using 0.275 million rays. As shown, the errors are distributed around the mean which reasonably well agrees with the Gaussian distribution. This observation has proven the stochastic nature of the developed numerical method. Given this and knowing the standard deviation the uncertainty in the solution can be calculated with certain probability.

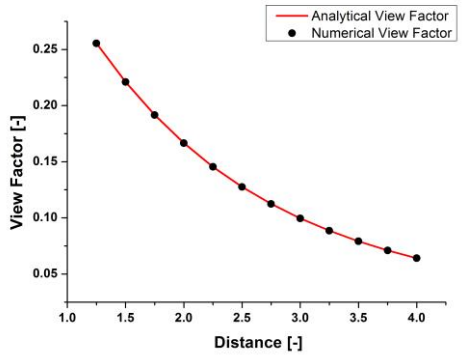

Figure 2. View factor vs. distance

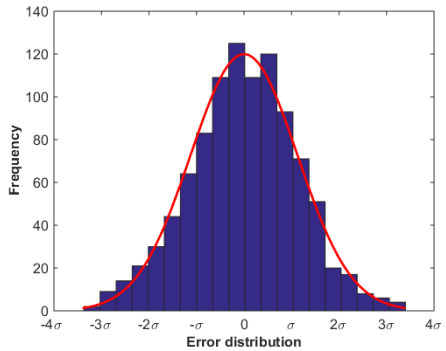

Figure 3. Histogram of relative error for a test composed of 1000 trials each using 0.275 million rays

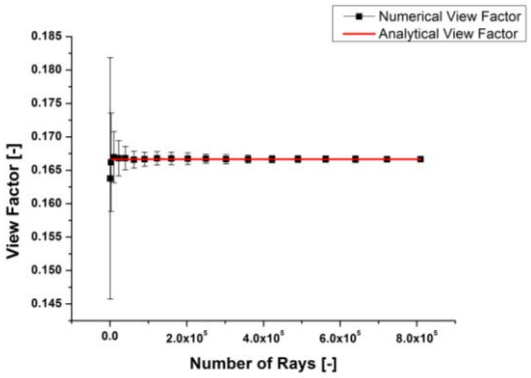

Figure 4a. Number of rays vs view factor with 2-σ error bars for a single distance

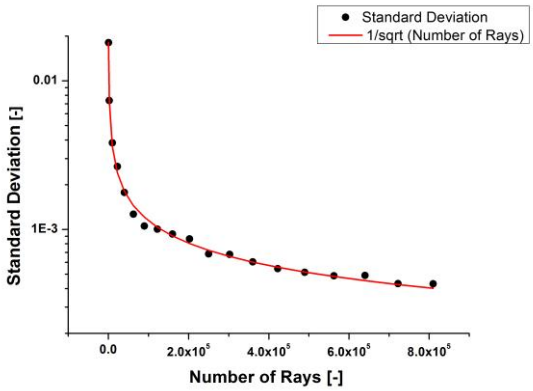

Figure 4b. Standard deviation vs number of rays

Figure 4a shows the mean numerical value of the calculated view factors based on one-hundred different calculations for a single distance value between the sphere and the plate. Each point in the figure corresponds to a different set of calculations with the indicated number of rays used. The vertical bars on top of the data points represent the standard deviation in the solutions. The heights of the bars are equal to twice the level (1 σ each side) of the standard distribution. As shown in Figure 4a, the higher the number of rays used in the calculations, the closer is the mean numerical value of the separate calculation sets to the analytical value shown with the continuous line. The other observation about these validation results is that the heights of the vertical bars, representing the magnitude equal twice the level of standard deviation, reduce with increasing number of rays used in the calculations.

The variation in the standard deviation with respect to the number of rays is shown in Figure 4b. According to the central limit theorem the standard deviation follows the relation $1/\sqrt{N}$ where N being the number of rays used in the Monte Carlo method. As shown in the figure, the reduction in the standard deviation is in analogous to the theory which again verifies the way the numerical method is implemented.

RESULTS AND DISCUSSION

As discussed in the numerical method section Monte Carlo ray tracing method requires tracing of about a quarter-of-million of rays to get a solution with acceptable accuracy. The resulting prohibitively large computational times can be substantially reduced with the use of the graphic processors and utilizing more efficient data storing techniques. The lack of dependency between the rays makes this method extremely suitable for the many-core architecture of the graphic processors that are capable of tracing thousands of rays in parallel. The present numerical tool has been developed on graphic processors and the processing speed is compared with similar ray tracing studies present in the literature.

Performance Gain purely by Computational Power of Graphic Cards

For the comparison of the computation time with the CPU, the scenarios studied by Mirhosseini and Saboonchi [10] and Cosson et al. [21] are selected. Table 1 shows the number of rays traced per second on the indicated hardware and as well as the year of publication. The past studies mentioned in the table are conducted on CPU whereas the present work is carried out on two different GPU's; one being a low-end laptop GPU while other one being a high-end GPU. As shown in Table 1 even with a low-end GPU the number of rays being traced could be increased by 15x. Moreover, the high end GPU is able to trace 30 million rays in a second which means about 1500x increase in the number of rays being traced compared with the recent studies shown in the table.

Table 1 : Comparison of the number of rays traced per second

Compared cases	Hardware	Number of rays traced per second (million)
Cosson et al. [21], 2011	Intel Centrino CPU (2.6 GHz)	0.02
Mirhosseini and Saboonchi [10], 2011	Not specified / CPU	0.007
Present work	Nvidia Geforce 610M GPU	0.28
Present work	Nvidia TitanX	30

However, it is important to indicate that the modern CPU hardware is much more powerful than the ones used in the past studies shown in the table. Using a modern high end CPU, it is estimated that the studies mentioned in the table would result in a 15-20x increase in the number of rays traced. [22] However, one should also consider that the computations on the CPU probably were conducted using double precision accuracy whereas in the current study on the GPU's single precision accuracy was used. Conducting the same computations on CPU using single precision accuracy would mean a further 2x increase in the number of rays traced. So, taking all into account it is estimated that the implementation of Cosson et. al. [21] would run about 30-40x on today's high end multi-core CPU's using single precision accuracy. This would mean that the current implementation still be better by factor of 40x in terms of the number of rays being traced. It is noteworthy to indicate that this achieved speed up is due to the computational power of GPU only. The performance of the ray tracing method can be further improved by using the so called acceleration data structures.

Performance Gain purely by the Acceleration Data Structures

The computational cost of the ray tracing calculations depends both on the number of rays being traced and on the number of the triangles present in the domain. In the naïve implementation the ray-triangle intersection calculations are carried out for each ray and for each triangle present in the domain regardless whether the ray will intersect or not. However, organizing the geometry data composed of triangles in a tree-like data structure can accelerate the intersection calculation massively. Different types of acceleration data structures exist but in this study the Bounding Volume Hierarchy (BVH), one of the most efficient acceleration data structures as discussed by Vinkler et al. [15], is considered. In order to investigate the effect of the BVH data structure on the performance, a view factor calculation study has been conducted. For this study the view factor test case shown in Figure 1 is considered where the number of triangles the plate is made of has been progressively increased without altering its dimensions. The computation time to trace 100 million ray has been recorded for each case.

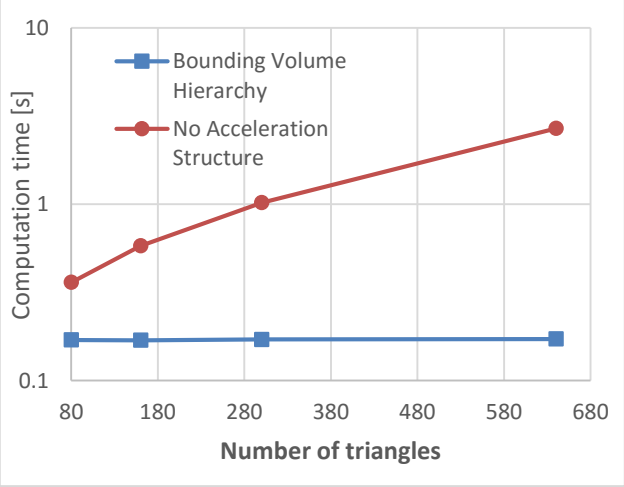

Figure 5. Number of triangles vs computation time to trace 100 million rays with and without acceleration structure

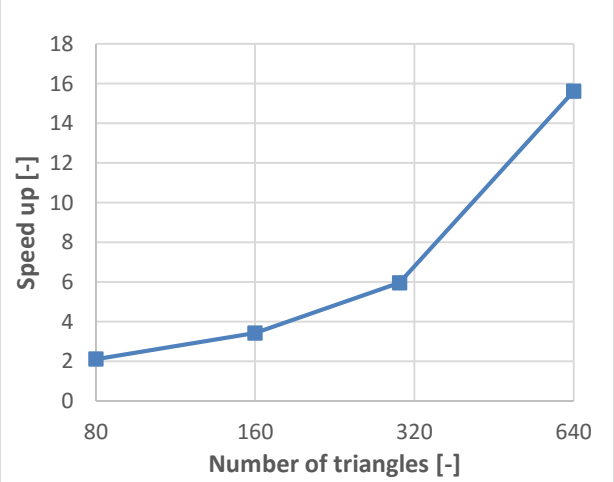

Figure 6. Speed up due to the acceleration structure vs number of triangles

As shown in Figure 5 there is an almost 8x increase in the computation time as the number of triangles was raised by 8x. On the other hand, the calculation carried out with the BVH acceleration data structures seems to be almost completely unaffected by the number of triangles present in the domain. Figure 6 shows the gained speed up entirely due to the usage of the BVH. As shown, the speed-up increases dramatically with the number of the triangles in the domain which highlights the importance of the acceleration data structures especially for large view factor calculations.

Combining the computational power of GPU and the acceleration data structure mentioned above very impressive speed-up values are achieved even with very low-end graphic cards. Table 2 shows the number of rays traced per second on the indicated hardware compared. As compared to the studies on CPU, the current one is able to trace around 300 and 700 times more rays within the same time frame compared to the studies in [21] and in [10] respectively with low end GPU. Moreover, the results become even more dramatic with the high end GPU showing a speed-up in excess of 10,000x.

Table 2: Comparison of the number of rays traced per second rays with acceleration structure

Compared cases	Hardware	Number of rays traced per second (million)
Cosson et al. [21], 2011	Intel Centrino CPU (2.6 GHz)	0.02
Mirhosseini and Saboonchi [10], 2011	Not specified / CPU	0.007
Present work	Nvidia Geforce 610M GPU	5.2
Present work	Nvidia TitanX	250

Implementation on Complex Geometry:

After the analysis on computation time and acceleration data structures, the numerical method is next tested for a scenario involving a complex three dimensional geometry. In this regard, the view factor calculations have been conducted between two glasses shown in Figure 7. The surfaces are represented by 7056 surface triangles. Rays are emitted continuously over the randomly chosen surface elements and the obtained results are plotted after indicated ray numbers given in Figure 7. For this scenario, a reasonably accurate result requires at least tracing 10 million rays which take around two seconds on the low performance laptop GPU and 0.04 seconds on high end GPU.

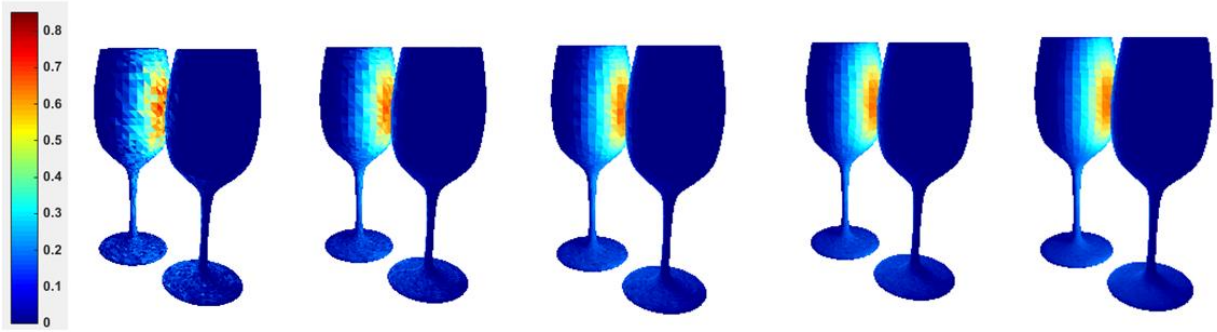

Figure 7. Map of view factor over the glass surface for different number of rays used (in millions left to right: 0.5 - 2.5 - 6.4 - 9.6 – 16). The hot points are becoming more clear and sharper with the increasing number of rays.

SUMMARY AND CONCLUSIONS

In this numerical study a Monte Carlo ray tracing based view factor calculation tool has been developed. The tool has been validated and its capability on conducting view factor calculations involving complex geometries has been demonstrated. Using the graphic processors computational cost of the method is substantially reduced. It is calculated that a high-end GPU offers a performance benefit of about 40x in comparison to today's high end multi-core CPU's. In the present study the importance of the acceleration data structure in ray tracing calculations is also demonstrated. In the scenarios considered in this work the benefit of using the acceleration data structures is calculated to be around 10x which might go well above that with the increase in the complexity of the model. Results of the present study reveals the potential of GPU computing in expanding the applicability of the ray tracing based numerical methods on the solution of the radiative transfer equation.

REFERENCES

1. J.R. Howell, *A Catalog of Radiation Configuration Factors*, McGraw-Hill, New York, 1982.
2. J.R. Howell and M.P. Mengüç, "The JQSRT Web-Based Configuration Factor Catalog: A Listing of Relations for Common Geometries", *Journal of Quantitative Spectroscopy and Radiative Transfer*, vol. 112, no. 5, pp. 910-912, March 2011.
3. On line Catalog, Thermalradiation.net, http://www.thermalradiation.net/indexCat.html
4. W. Nusselt, "Graphische bestimmung des winkelverhaltnisses bei der wärmestrahlung", *Zeitschrift des Vereines Deutscher Ingenieure*, vol. 72, no. 20, pp. 673, 1928.
5. R. Farrel , "Determination of Configuration Factors of Irregular Shape" , *Trans. ASME J. Heat Transfer* , vol. 98 , pp. 311 – 313 , 1976.
6. H. C. Hottel, "Radiant heat transmission," *Heat Transmission, ed. W. H. McAdams*, Third Edition., McGraw- Hill, New York, 1954.

7. J. R. Howell, “The Monte Carlo Method in Radiative Transfer,” *J. Heat Transfer*, vol. 120, pp. 547–560, 1998.
8. A. F. Emery, O. Johansson, M. Lobo, and A. Abrous, “A comparative study of methods for computing the diffuse radiation view factors for complex structures.” *J. Heat Transfer*, vol. 113, pp. 413–421, 199.
9. A. R. Hajji, M. Mirhosseini, A. Saboonchi, A. Moosavi, “Different methods for calculating a view factor in radiative applications: Strip to in-plane parallel semi-cylinder” *J. Engineering Thermophysics*, vol. 24, no. 2, pp. 169-180, 2015.
10. M. Mirhosseini, A. Saboonchi, “View factor calculation using the Monte Carlo method for a 3D strip element to circular cylinder,” *Int. Commun. Heat Mass Transfer*, vol. 36, no. 6, pp. 821–826, 2011.
11. T. Whitted, “An improved illumination model for shaded display,” *Communications of the ACM*, vol. 23, no. 6, pp. 343-349,1980.
12. J.T. Kajiya, "The rendering equation," *Proceedings of the 13th annual conference on Computer graphics and interactive techniques*, 1986.
13. J. L. Bentley, “Multidimensional binary search trees used for associative searching”, *Communications of the ACM*, vol.18 no.9, pp.509-517, 1975.
14. C. Ericson, *Real-Time Collision Detection*. 2004. pp 236-237.
15. M.Vinkler, V. Havran, J. Bittner, “Bounding volume hierarchies versus kd-trees on contemporary many-core architectures”, *Proceedings of the 30th Spring Conference on Computer Graphics*, May 28-30, 2014.
16. S. G. Parker, J. Bigler, A. Dietrich, H. Friedrich, J. Hoberock, D. Luebke, D. McAllister, M. McGuire, K. Morley, A. Robison, M. Stich, “OptiX: A general purpose ray tracing engine,” ACM Transactions on Graphics (TOG), vol. 29, no. 4, July 2010.
17. S. Halverson, *Energy Transfer Ray Tracing with OptiX*. Master’s Thesis. University of Minnesota, 2012.
18. J.R. Howell, M. P. Mengüç, R. Siegel, *Thermal Radiation Heat Transfer,* 6th Edition, Taylor and Francis, CRC Press, New York, 2016.
19. F. Zafar, M. Olano , A. Curtis, “GPU random numbers via the tiny encryption algorithm,” *Proceedings of the Conference on High Performance Graphics*, Saarbrucken, Germany, 2010.
20. A. Feingold and K.G. Gupta, "New Analytical Approach to the Evaluation of Configuration Factors in Radiation from Spheres and Infinitely Long Cylinders," *Journal of Heat Transfer*, vol. 92, pp. 69-76, Feb. 1970.
21. B. Cosson, F. Schmidt, Y. Le Maoult, M. Bordival. “Infrared heating stage simulation of semi-transparent media (PET) using ray tracing method*,” International Journal of Material Forming*, 2010,” vol. 4, pp. 1-10, 2011.
22. https://www.cpubenchmark.net/